

LLM-Driven Adaptive Cloud Resource Scheduling: Bridging Reasoning Intelligence With Optimization Guarantees

GUANYU DING ¹, SHIYU YANG ², HAN LIN ³ (Member, IEEE), ZIFAN CHEN ⁴, AND JIE SI YANG ⁵¹New York University, New York, NY 10012 USA²University of California, Los Angeles, Los Angeles, CA 90095 USA³University of Wisconsin–Madison, Madison, WI 53706 USA⁴University of Pennsylvania, Philadelphia, PA 19104 USA⁵The University of Utah, Salt Lake City, UT 84112 USA

CORRESPONDING AUTHOR: GUANYU DING (e-mail: gd2292@nyu.edu).

ABSTRACT Cloud resource scheduling presents a fundamental challenge in modern distributed computing, where heterogeneous workloads, complex task dependencies, and multi-objective optimization requirements exceed the capabilities of traditional rule-based and small-scale machine learning approaches. Existing schedulers struggle to dynamically adapt to evolving workload patterns while simultaneously satisfying Service Level Agreement (SLA) constraints, resource efficiency targets, and fairness policies. This article introduces *LLMSched*, a novel Large Language Model (LLM)-driven adaptive scheduling framework that synergistically combines the contextual reasoning capabilities of foundation models with the execution guarantees of classical optimization techniques. Our approach transforms heterogeneous cluster states—including task dependency graphs, real-time resource utilization metrics, and SLA specifications—into a unified structured-textual representation that leverages LLMs’ few-shot learning and causal reasoning abilities to generate intelligent scheduling candidates. These candidates are subsequently refined through a lightweight Integer Linear Programming (ILP) module that ensures feasibility and optimality under resource constraints. We evaluate *LLMSched* on Google’s production cluster trace dataset, demonstrating significant improvements over state-of-the-art baselines: 23.7% reduction in average job completion time, 18.4% improvement in resource utilization efficiency, and 31.2% decrease in SLA violations across diverse workload scenarios. Extensive ablation studies validate the contributions of each architectural component, while robustness analysis under workload perturbations confirms the framework’s practical viability. Our work establishes a new paradigm for intelligent resource management that bridges the gap between neural reasoning and algorithmic precision, opening avenues for LLM applications in systems optimization domains.

INDEX TERMS Cloud computing, resource scheduling, large language models (LLMs), adaptive optimization, task orchestration, SLA management.

I. INTRODUCTION

Cloud computing has fundamentally reshaped modern information technology by enabling on-demand access to large-scale computational resources for applications ranging from web services to scientific computing [1], [2]. As cloud infrastructures continue to scale to millions of servers across geographically distributed datacenters [3], efficient resource scheduling has become a critical challenge affecting both operational cost and quality of service. Modern cloud

platforms must coordinate heterogeneous workloads—including latency-sensitive microservices, batch analytics, and machine learning tasks—each with distinct resource demands and Service Level Agreement (SLA) requirements [4], [5].

Traditional cloud schedulers primarily rely on hand-crafted heuristics and rule-based policies [6], [7], [8]. While these approaches offer predictability and low overhead, they struggle to cope with the growing complexity of modern workloads.

In particular, static heuristics fail to capture intricate task dependencies and data locality constraints [9], adapt to temporal workload variations [10], or balance multiple conflicting objectives such as efficiency, fairness, and latency [11]. These limitations often result in suboptimal resource utilization under dynamic conditions.

Recent advances in machine learning have motivated learning-based scheduling approaches, especially those based on deep reinforcement learning (DRL) [12], [13], [14]. While such methods show promising results in controlled environments, their deployment in production systems remains challenging. DRL schedulers require extensive training data and carefully designed reward functions, making them brittle under workload distribution shifts [15]. Moreover, the black-box nature of neural policies hinders interpretability and debugging, which are essential for mission-critical cloud systems [16]. Existing models also rely on low-dimensional numerical state representations, discarding rich semantic information available in task descriptions, dependency graphs, and system logs [17].

The emergence of Large Language Models (LLMs), such as GPT-4 [18], Claude [19], and LLaMA [20], has demonstrated strong capabilities in reasoning, planning, and knowledge synthesis [21]. These models offer several properties attractive for scheduling, including contextual understanding of heterogeneous inputs [22], few-shot adaptability [23], chain-of-thought reasoning [24], and semantic interpretation of natural language constraints [25]. Early studies have explored their potential in code generation, system configuration, and workflow optimization [26], [27], [28], suggesting promise for systems-level decision making.

However, directly applying LLMs to cloud scheduling poses fundamental challenges. Scheduling decisions must satisfy hard constraints that cannot be guaranteed by probabilistic language model outputs [29]. Furthermore, the inference latency of LLMs is incompatible with the high-throughput, low-latency requirements of production schedulers [30]. LLMs may also generate syntactically valid yet semantically incorrect plans that violate system invariants [31]. These issues highlight the need for a hybrid design that leverages LLM reasoning while preserving algorithmic correctness and efficiency.

In this article, we propose LLMsSched, a hybrid framework that integrates LLM-driven high-level reasoning with classical optimization for adaptive cloud resource scheduling. Rather than using LLMs as end-to-end decision makers, LLMsSched positions them as policy generators that explore promising regions of the solution space, while delegating constraint enforcement and fine-grained optimization to algorithmic modules. The framework consists of three stages. (1) Structured-Textual State Representation encodes heterogeneous cluster information, including task DAGs, resource metrics, historical outcomes, and SLA specifications, into a compact textual format suitable for LLM reasoning. (2) LLM-Guided Candidate Generation employs few-shot and chain-of-thought prompting to produce diverse scheduling

candidates that consider data locality, load balancing, and task priorities. (3) Optimization-Based Refinement uses a lightweight Integer Linear Programming (ILP) solver to enforce hard constraints and optimize a multi-objective utility function, ensuring correctness and robustness. Especially, to mitigate inference overhead, LLMsSched incorporates caching and workload-aware triggering mechanisms that invoke LLM reasoning only when significant state changes occur. For routine scheduling decisions, validated strategies are executed by fast heuristic modules, achieving sub-millisecond latency while retaining adaptability.

Our contributions are summarized as follows: (1) We introduce the first LLM-driven cloud scheduling framework that systematically combines foundation model reasoning with classical optimization. (2) We design a structured-textual state representation and constraint-guided prompting strategies that enable interpretable and effective LLM-based scheduling. (3) We develop a hybrid optimization architecture that guarantees scheduling correctness through lightweight ILP refinement with practical overhead. (4) We conduct extensive evaluations on large-scale production traces, demonstrating consistent improvements over diverse baselines and providing insights into when and why LLM-guided scheduling is effective.

II. RELATED WORKS

Traditional Cloud Scheduling Systems: Cloud scheduling has evolved from early batch systems such as Condor [32] and Sun Grid Engine to large-scale cluster managers designed for heterogeneous, multi-tenant workloads. Google's Borg [5] established a production-grade centralized scheduling architecture with admission control, priority-based preemption, and bin-packing, while Kubernetes [33], [34] operationalized these ideas for container orchestration via extensible, heuristic-driven schedulers. Alternative paradigms include two-level schedulers that delegate decisions to application frameworks (e.g., Mesos [7], YARN [8]) and optimistic concurrency control for parallel scheduling (Omega [6]). Recent work further explores fairness- and resource-aware policies, such as DRF [35], multi-dimensional bin packing (Tetris [36]), and specialized accelerators scheduling (Carbyne [37]). Despite their maturity and robustness, these systems rely predominantly on manually designed heuristics and objective-specific policies, which limits their ability to adapt to complex, evolving workload semantics.

Learning-Based Scheduling Approaches: Recent years have seen extensive efforts to apply learning-based methods to cluster and resource scheduling. Early work such as DeepRM [12] formulates scheduling as an MDP and demonstrates that deep reinforcement learning can outperform heuristic policies. Subsequent systems explore richer models and learning paradigms, including GNN-based dependency modeling for DAG jobs (Decima [38]), performance prediction via Bayesian optimization (Auto-Tune [39]), temporal workload modeling with RNNs (NNS [40]), and multi-agent reinforcement learning for distributed scheduling (Chronus [41]). Despite promising results, learning-based

schedulers suffer from practical limitations, including heavy reliance on representative training traces [42], sensitivity to distribution shift [43], limited interpretability [44], and restricted state representations that overlook high-level semantic information [45]. To mitigate these issues, hybrid systems combine learning with optimization, using learned models for demand or cost estimation while relying on mathematical solvers for decision making (e.g., Cilantro [46], Harmony [47]). Our work follows this hybrid philosophy and further extends it by leveraging the reasoning capabilities of large language models.

Large Language Models in Systems Large language models have recently been explored beyond NLP, first demonstrating strong capabilities in code generation and software engineering tasks (e.g., Codex [26], AlphaCode [48]), with broad adoption in developer workflows. Building on this success, a growing body of work investigates LLMs for systems problems, including failure diagnosis (D-Bot [49]), control and optimization via natural-language reasoning (LLMLight [50]), and database query optimization (xTuring [51]). LLMs have also been applied to system configuration and tuning, such as generating Kubernetes parameters (LLMAO [27]), recommending Spark configurations (Config-GPT [52]), and automating database administration tasks (AutoAdmin [53]). Despite promising results, these systems largely operate in offline or advisory settings, with human validation prior to deployment. Direct use of LLMs for real-time or safety-critical control remains limited due to fundamental challenges, including lack of hard constraint guarantees [29], high inference latency [54], and susceptibility to hallucinations in numerical or low-level reasoning [55]. Recent work therefore advocates hybrid paradigms that restrict LLMs to semantic understanding and heuristic generation while delegating constraint satisfaction to formal solvers, as exemplified by LLM-modulo frameworks [31]. Our work follows this line of thinking by explicitly separating LLM reasoning from algorithmic guarantees.

III. PRELIMINARIES

In this section, we formalize the cloud resource scheduling problem and establish the mathematical foundations for our LLM-driven approach. We define key concepts including cluster models, task representations, resource constraints, and optimization objectives that guide our system design.

A. CLUSTER MODEL

We model a cloud cluster as a set of heterogeneous compute nodes $\mathcal{N} = \{n_1, n_2, \dots, n_N\}$, where each node n_i is characterized by its resource capacity vector $\mathbf{c}_i = [c_i^{\text{CPU}}, c_i^{\text{MEM}}, c_i^{\text{DISK}}, c_i^{\text{NET}}]^\top \in \mathbb{R}_+^4$, representing CPU cores, memory (GB), disk bandwidth (MB/s), and network bandwidth (Gbps), respectively. At any time t , each node maintains a current resource utilization state $\mathbf{u}_i(t) \in [0, 1]^4$, where $u_i^r(t)$ denotes the fraction of resource type r currently allocated on node n_i .

Nodes may belong to different availability zones $\mathcal{Z} = \{z_1, \dots, z_K\}$ with varying reliability characteristics and network latencies. We denote the zone assignment function as $\zeta : \mathcal{N} \rightarrow \mathcal{Z}$, and define inter-zone latency matrix $\mathbf{L} \in \mathbb{R}_+^{K \times K}$, where L_{jk} represents the average network latency between zones z_j and z_k .

B. TASK AND JOB REPRESENTATION

Cloud workloads consist of *jobs*, each comprising one or more interdependent *tasks*. We represent a job J as a directed acyclic graph (DAG) $G_J = (\mathcal{T}_J, \mathcal{E}_J)$, where: $\mathcal{T}_J = \{t_1, \dots, t_{|\mathcal{T}_J|}\}$ is the set of tasks belonging to job J ; $\mathcal{E}_J \subseteq \mathcal{T}_J \times \mathcal{T}_J$ represents precedence constraints, where $(t_i, t_j) \in \mathcal{E}_J$ indicates that task t_j cannot start until task t_i completes.

Each task $t \in \mathcal{T}_J$ is characterized by: 1) Resource demand: $\mathbf{d}_t = [d_t^{\text{CPU}}, d_t^{\text{MEM}}, d_t^{\text{DISK}}, d_t^{\text{NET}}]^\top \in \mathbb{R}_+^4$; 2) Execution time: $e_t : \mathcal{N} \rightarrow \mathbb{R}_+$, representing the estimated duration on each node type; 3) Data dependencies: $\mathcal{D}_t = \{(t', s_t)\}$, where t' is a predecessor task and $s_t \in \mathbb{R}_+$ is the data size transferred from t' to t ; 4) Priority level: $p_t \in \{1, 2, \dots, P\}$, where higher values indicate greater urgency.

Jobs are submitted continuously over time. Let $\mathcal{J}(t)$ denote the set of active jobs at time t , comprising both running and pending jobs. Each job J has an arrival time a_J , a deadline d_J (for latency-sensitive jobs), and a weight $w_J \in \mathbb{R}_+$ reflecting its business importance.

C. SCHEDULING DECISION AND CONSTRAINTS

A scheduling decision at time t is defined as a mapping function $\sigma_t : \mathcal{T} \rightarrow \mathcal{N} \cup \{\perp\}$, where $\mathcal{T} = \bigcup_{J \in \mathcal{J}(t)} \mathcal{T}_J$ is the set of all active tasks, and $\perp$ represents the decision to defer scheduling (task remains pending). To ensure that a scheduling decision is executable in a real cluster, it must satisfy a set of hard feasibility constraints derived from resource availability and job execution semantics.

Resource Capacity Constraint: At each decision epoch t , every node has only limited *residual* resources due to previously scheduled tasks. We denote by c_i^r the total capacity of resource type r on node n_i , and by $u_i^r(t) \in [0, 1]$ the fraction of that resource already in use at time t . A task can only be assigned to a node if its resource demands can be fully accommodated by the node's remaining capacity:

$$\sum_{t \in \mathcal{T} : \sigma_t(t) = n_i} d_t^r \leq c_i^r \cdot (1 - u_i^r(t)), \quad (1)$$

$$\forall i \in [N], r \in \{\text{CPU}, \text{MEM}, \text{DISK}, \text{NET}\}.$$

This prevents over-allocation and guarantees that all assigned tasks can be executed without violating resource limits.

Precedence Constraint: A task can only be scheduled if all its predecessors have completed:

$$\sigma_t(t_j) \neq \perp \Rightarrow \forall (t_i, t_j) \in \mathcal{E}_J : \text{completed}(t_i, t). \quad (2)$$

This constraint guarantees correct execution order and preserves data and control dependencies.

Affinity Constraint: Certain tasks may have location preferences, expressed as a set of allowed nodes $\mathcal{N}_t \subseteq \mathcal{N}$:

$$\sigma_t(t) \neq \perp \Rightarrow \sigma_t(t) \in \mathcal{N}_t. \quad (3)$$

For each task t , the set $\mathcal{N}_t \subseteq \mathcal{N}$ specifies the nodes on which the task is allowed to run, reflecting practical requirements such as hardware specialization, data locality, or geographic restrictions. If a task is scheduled, it must be placed on one of the nodes in $\mathcal{N}_t$.

Anti-Affinity Constraint: For fault tolerance, replicated tasks of the same job should be placed on different nodes:

$$\forall t_i, t_j \in \mathcal{R}_J : i \neq j \Rightarrow \sigma_t(t_i) \neq \sigma_t(t_j), \quad (4)$$

where $\mathcal{R}_J \subseteq \mathcal{T}_J$ denotes the set of replicated tasks in job J . This ensures that tasks of the same group (e.g., replicas of the same service) are not co-located on the same node.

D. PERFORMANCE METRICS AND OBJECTIVES

We evaluate scheduling quality using several metrics that reflect key operational objectives in large-scale cloud systems. Intuitively, an effective scheduler should complete jobs quickly, utilize cluster resources efficiently, satisfy deadline-driven service-level objectives (SLOs), and maintain fairness across jobs or users. We formally define these metrics below:

Job Completion Time (JCT): The total time from job submission to completion. For job J scheduled according to policy π , the completion time is:

$$\text{JCT}_\pi(J) = C_J - a_J, \quad (5)$$

where C_J is the completion time of the last task in J . The average weighted JCT across all jobs is:

$$\text{Avg-JCT}_\pi = \frac{1}{\sum_J w_J} \sum_{J \in \mathcal{J}} w_J \cdot \text{JCT}_\pi(J). \quad (6)$$

Resource Utilization: The fraction of cluster resources actively used. For resource type r at time t :

$$U^r(t) = \frac{\sum_{i=1}^N c_i^r \cdot u_i^r(t)}{\sum_{i=1}^N c_i^r}. \quad (7)$$

The time-averaged utilization is:

$$\bar{U}^r = \frac{1}{T} \int_0^T U^r(t) dt. \quad (8)$$

SLA Violation Rate: The proportion of deadline-sensitive jobs that miss their deadlines:

$$\text{SVR}_\pi = \frac{|\{J \in \mathcal{J} : C_J > d_J\}|}{|\{J \in \mathcal{J} : d_J < \infty\}|}. \quad (9)$$

Fairness: We adopt the Jain's fairness index to measure allocation equity across users. Let ϕ_u denote the resource share received by user u normalized by their demand:

$$\text{Fairness} = \frac{\left(\sum_{u=1}^U \phi_u\right)^2}{U \cdot \sum_{u=1}^U \phi_u^2}. \quad (10)$$

Makespan: For batch jobs, we measure the total time to complete all jobs in a workload $\mathcal{W}$:

$$\text{Makespan}_\pi(\mathcal{W}) = \max_{J \in \mathcal{W}} C_J - \min_{J \in \mathcal{W}} a_J. \quad (11)$$

E. MULTI-OBJECTIVE OPTIMIZATION PROBLEM

The cloud scheduling problem can be formulated as a multi-objective optimization:

$$\begin{aligned} \min_{\pi} \mathcal{L}(\pi) = & \alpha_1 \cdot \text{Avg-JCT}_\pi + \alpha_2 \cdot (1 - \bar{U}) + \alpha_3 \cdot \text{SVR}_\pi \\ & + \alpha_4 \cdot (1 - \text{Fairness}) + \alpha_5 \cdot \text{Makespan}_\pi, \end{aligned} \quad (12)$$

subject to constraints (1)–(4), where $\alpha_i \geq 0$ are weights reflecting the relative importance of each objective, and $\sum_{i=1}^5 \alpha_i = 1$. The scalarized objective in (12) aggregates multiple, potentially conflicting goals into a single utility function. This formulation captures the fundamental tension in cloud scheduling: minimizing completion times often requires aggressive resource allocation that may compromise fairness; maximizing utilization may delay low-priority jobs and violate SLAs; achieving all objectives simultaneously under dynamic workload arrivals is computationally intractable for large-scale clusters [56].

IV. METHODOLOGY

In this section, we present the architecture and algorithmic components of LLMSched. We begin with a system overview, then detail each module: structured-textual state encoding, LLM-guided candidate generation with constraint-aware prompting, optimization-based refinement, and efficient execution strategies. We emphasize the design motivations and technical innovations that enable practical deployment.

A. SYSTEM ARCHITECTURE OVERVIEW

The LLMSched architecture consists of four interconnected modules operating in a closed-loop feedback cycle:

- 1) *State Encoder*: Transforms heterogeneous cluster information into structured-textual representations suitable for LLM consumption
- 2) *LLM Reasoning Engine*: Generates diverse scheduling candidates through carefully designed prompting strategies
- 3) *ILP Refinement Module*: Validates and optimizes LLM-generated candidates to ensure constraint satisfaction
- 4) *Execution Controller*: Manages scheduling decisions, monitors cluster state, and triggers LLM invocations adaptively

The system operates in two modes: *intelligent mode* (invokes LLM for complex scenarios) and *fast mode* (executes cached strategies for routine decisions). This hybrid execution strategy balances reasoning quality with computational efficiency.

Design Motivation: Traditional schedulers make decisions in milliseconds but lack adaptability; LLM inference requires hundreds of milliseconds but provides superior reasoning. Our architecture leverages LLMs as strategic planners for high-level policies while delegating tactical execution to

lightweight algorithms, achieving both intelligence and responsiveness.

B. STRUCTURED-TEXTUAL STATE ENCODING

1) ENCODING RATIONALE

LLMs excel at processing natural language but struggle with raw numerical data and graph structures. Conversely, traditional ML schedulers use low-dimensional feature vectors that discard semantic information. Our encoding scheme bridges this gap by creating a *hybrid representation* that preserves both numerical precision and semantic richness.

2) MULTI-LEVEL ENCODING STRATEGY

We decompose cluster state into four hierarchical levels, each encoded differently:

Level 1. Cluster-Wide Summary: We generate a concise textual summary of overall cluster health:

```
CLUSTER STATE [Time: 2025-10-09 14:23:15]
Total Nodes: 1000 | Active: 987 | Degraded: 13
CPU Utilization: 67.3% | Memory: 72.1%
Pending Jobs: 234 | Running Jobs: 1876
High-Priority Queue: 45 jobs | SLA Violations (1h): 12
```

This high-level context helps the LLM understand the current operational regime (normal, congested, under-utilized).

Level 2. Resource Availability Matrix: For each node, we encode availability as a structured table. To manage context limits, we employ *hierarchical clustering*: nodes are grouped by resource profiles, and only cluster representatives are detailed:

$$\text{NodeCluster}(n_i) = \arg \min_{k \in [K]} \|c_i - \mu_k\|_2, \quad (13)$$

where μ_k is the centroid of cluster k . We then encode:

```
RESOURCE CLUSTERS (5 clusters, 200 nodes each)
C1 [High-CPU]: CPU=32 cores (avg avail: 12.4)
    MEM=128 GB (avg avail: 45.2 GB) | Zone: us-west-1a
C2 [Memory-Intensive]: CPU=16 (avail: 8.1)
    MEM=256 GB (avail: 98.7 GB) | Zone: us-west-1b
...
```

Level 3. Task Dependency Graphs: For each pending job, we encode its DAG structure using a custom textual format that preserves precedence relationships:

```
JOB J_1234 [Priority: HIGH, Deadline: 180s,
Weight: 2.5]
T1: Map [CPU:4, MEM:8 GB, EST:30s] → {T3, T4}
T2: Map [CPU:4, MEM:8 GB, EST:32s] → {T3, T4}
T3: Reduce [CPU:8, MEM:16 GB, EST:45s] → {T5}
    Data from T1: 2.3GB
    Data from T2: 2.1GB
T4: Reduce [CPU:8, MEM:16 GB, EST:43s] → {T5}
T5: Aggregate [CPU:2, MEM:4 GB, EST:15s] → END
    Data from T3: 0.8 GB, T4: 0.9GB
```

This representation explicitly shows parent-child relationships and data transfer volumes, enabling the LLM to reason about data locality and parallelism opportunities.

Level 4. Constraint Specifications: We translate operational constraints into natural language augmented with structured fields:

CONSTRAINTS:

1. Affinity: Job J_1234 tasks must be in zone us-west-1
2. Anti-Affinity: Replicated tasks T1, T2 on different nodes
3. SLA: Job J_1234 deadline in 180s (current: 165s remaining)
4. Resource Reservation: Node N_567 reserved for user U_89

3) CONTEXT WINDOW MANAGEMENT

Given LLM context limits (e.g., 128 K tokens for GPT-4), we cannot encode full cluster state for large deployments. We implement a *relevance-based filtering* mechanism:

$$\begin{aligned} \text{Relevance}(J, n_i) = & \beta_1 \cdot \text{ResourceMatch}(J, n_i) \\ & + \beta_2 \cdot \text{LocalityScore}(J, n_i) \\ & + \beta_3 \cdot \text{HistoricalAffinity}(J, n_i), \end{aligned} \quad (14)$$

where:

- $\text{ResourceMatch}(J, n_i) = \exp(-\|d_J - c_i^{\text{avail}}\|_2)$ measures resource fit,
- $\text{LocalityScore}(J, n_i)$ reflects data locality based on predecessor task placements,
- $\text{HistoricalAffinity}(J, n_i)$ captures learned preferences from past executions.

We select the top- M most relevant nodes for each job, where M is dynamically adjusted to fit context limits while ensuring sufficient scheduling options.

Design Motivation: This multi-level encoding preserves the semantic structure that LLMs leverage during pre-training (hierarchies, dependencies, natural language specifications) while maintaining computational tractability. Unlike pure numerical embeddings, our representation remains human-interpretable, facilitating debugging and operator trust.

C. LLM-GUIDED CANDIDATE GENERATION

1) PROMPTING STRATEGY DESIGN

We employ a carefully crafted prompting pipeline that guides the LLM through the scheduling reasoning process:

System Prompt: Establishes the LLM's role and operational context:

You are an expert cloud resource scheduler managing a large-scale distributed cluster. Your goal is to assign pending tasks to compute nodes while optimizing job completion time, resource utilization, and SLA compliance. Consider task dependencies, data locality, resource constraints, and fairness across users.

Few-Shot Examples: We provide 3-5 carefully selected exemplars demonstrating effective scheduling decisions:

Example 1: For a high-priority MapReduce job with tight deadline, the scheduler co-located map tasks near input data and placed reduce tasks on high-memory nodes to avoid swapping...

These examples encode domain best practices (data locality, load balancing, priority handling) that guide the LLM's reasoning.

Chain-of-Thought Prompting: We explicitly request step-by-step reasoning:

Before providing task assignments, reason through: (1) Which jobs are most urgent? (2) What are the critical resource bottlenecks? (3) How can we minimize data movement? (4) Are there opportunities for task co-location?

This structured reasoning improves decision quality and provides interpretability.

2) CANDIDATE GENERATION PROCESS

Given the encoded state S_t , we query the LLM multiple times with varying temperatures to generate a diverse candidate set $\mathcal{C} = \{c_1, \dots, c_K\}$. We use temperatures $\tau \in [0.3, 0.8]$ to balance between conservative (low τ) and exploratory (high τ) strategies. Lower temperatures are used for high-priority jobs requiring reliable decisions, while higher temperatures enable creative solutions for complex scenarios.

3) CONSTRAINT-GUIDED GENERATION

To improve the feasibility of LLM outputs, we augment prompts with explicit constraint reminders:

$P_{\text{constrained}} = P_{\text{base}} \oplus$ "CRITICAL: Task T3 requires ≥ 16 GB memory. Only nodes N_45, N_78, N_102 satisfy this."

For each task with strict requirements, we inject specific constraint reminders into the prompt. This *constraint-aware prompting* significantly reduces invalid assignments (from 34% to 8% in our experiments).

Design Motivation: Directly applying LLMs without guidance produces syntactically valid but semantically poor schedules. Our prompting strategy injects domain expertise through few-shot examples and structures the reasoning process through chain-of-thought, mimicking how human schedulers approach complex decisions. Temperature diversity ensures solution exploration while constraint-guided generation improves feasibility.

D. OPTIMIZATION-BASED REFINEMENT

1) MOTIVATION FOR HYBRID OPTIMIZATION

LLM-generated candidates provide intelligent starting points but may violate hard constraints or exhibit suboptimal resource packing. We employ Integer Linear Programming (ILP) to refine candidates while preserving their high-level strategic decisions.

2) ILP FORMULATION

Let $x_{tj} \in \{0, 1\}$ be a binary variable indicating whether task t is assigned to node n_j . The refinement problem is formulated

as:

$$\min_x \sum_{t \in \mathcal{T}} \sum_{j=1}^N w_t \cdot e_t(n_j) \cdot x_{tj} + \lambda \cdot \sum_{t \in \mathcal{T}} \sum_{j=1}^N d_{\text{LLM}}(t, n_j) \cdot x_{tj}, \quad (16)$$

subject to:

$$\sum_{j=1}^N x_{tj} = 1, \quad \forall t \in \mathcal{T}, \quad (17)$$

$$\sum_{t \in \mathcal{T}} d_t^r \cdot x_{tj} \leq c_j^r \cdot (1 - u_j^r), \quad \forall j, r, \quad (18)$$

$$x_{t_i j} \cdot C_{t_i} \leq S_{t_k}, \quad \forall (t_i, t_k) \in \mathcal{E}, j, \quad (19)$$

$$x_{tj} = 0, \quad \forall t, j : n_j \notin \mathcal{N}_t, \quad (20)$$

where (16) minimizes weighted completion time plus deviation from LLM suggestions, $d_{\text{LLM}}(t, n_j) = 0$ if LLM suggested assigning t to n_j , else $d_{\text{LLM}}(t, n_j) = 1$, λ controls the trade-off between optimization and LLM guidance (we use $\lambda = 0.3$), (17) ensures each task is assigned exactly once, (18) enforces resource capacity, (19) maintains precedence constraints (C_{t_i} : completion time, S_{t_k} : start time), and (20) respects affinity constraints.

3) LIGHTWEIGHT SOLVING STRATEGY

Full ILP solving is expensive for large problem instances. We employ several acceleration techniques:

Warm Starting: Initialize the ILP solver with the LLM candidate as a feasible starting solution (after minimal corrections). This reduces solving time by 60-80%.

Time-Limited Solving: We impose a strict time budget (50 ms) on the ILP solver. If optimality is not reached, the best feasible solution found is returned. This ensures bounded latency.

Decomposition: For very large jobs, we decompose the problem into independent sub-problems based on DAG structure:

$$\mathcal{T} = \mathcal{T}_1 \cup \mathcal{T}_2 \cup \dots \cup \mathcal{T}_p, \quad (21)$$

where $\mathcal{T}_i$ are weakly connected components. Each sub-problem is solved independently in parallel, substantially reducing computational overhead while preserving optimality.

Constraint Pruning: We eliminate obviously infeasible assignments before ILP formulation:

$$\mathcal{F}(t) = \{n_j : d_t^r \leq c_j^r \cdot (1 - u_j^r), \forall r\}, \quad (22)$$

restricting x_{tj} to $j \in \mathcal{F}(t)$, reducing problem size by $\sim 40\%$. Nodes that cannot satisfy the task's resource demands under the current utilization state are excluded in advance, reducing the number of decision variables and accelerating the ILP solver without sacrificing feasibility.

Design Motivation: Pure optimization approaches struggle with semantic reasoning, while pure LLM outputs lack correctness guarantees. Our hybrid formulation uses the LLM

candidate as a “soft constraint” (via λ term), guiding optimization toward intelligent solutions while maintaining feasibility. The time-limited solving ensures predictable latency, critical for production deployment.

E. INTELLIGENT EXECUTION AND CACHING

1) WORKLOAD-AWARE TRIGGERING

Invoking the LLM for every scheduling decision is computationally prohibitive. We develop a triggering policy that invokes the LLM only when significant cluster state changes occur:

$$\text{Trigger}_{\text{LLM}} = \begin{cases} \text{True}, & \text{if } \Delta(S_t, S_{t-1}) > \theta \\ \text{False}, & \text{otherwise,} \end{cases} \quad (23)$$

where $\Delta(S_t, S_{t-1})$ measures state change magnitude:

$$\begin{aligned} \Delta(S_t, S_{t-1}) = & \alpha \cdot \frac{\|\mathbf{U}_t - \mathbf{U}_{t-1}\|_2}{\|\mathbf{U}_{t-1}\|_2} + \beta \cdot \frac{|\mathcal{J}_t \Delta \mathcal{J}_{t-1}|}{|\mathcal{J}_{t-1}|} \\ & + \gamma \cdot \mathbb{I}_{\text{SLA-risk}}. \end{aligned} \quad (24)$$

Here, $\mathbf{U}_t$ is the cluster utilization vector, $\mathcal{J}_t \Delta \mathcal{J}_{t-1}$ denotes symmetric difference in job sets, and $\mathbb{I}_{\text{SLA-risk}}$ is an indicator for approaching SLA violations. We set $\theta = 0.15$ based on sensitivity analysis.

2) STRATEGY CACHING AND REUSE

When the LLM is not triggered, we execute cached strategies from previous LLM invocations. We maintain a *strategy library* $\mathcal{L} = \{(S_i, \sigma_i, Q_i)\}$, where:

- S_i is a past cluster state (abstracted to resource availability profile),
- σ_i is the corresponding scheduling strategy,
- Q_i is the quality score (achieved objective value).

For a new state S_t , we retrieve the most similar cached strategy:

$$\sigma_{\text{cached}} = \arg \max_{\sigma_i \in \mathcal{L}} \text{Similarity}(S_t, S_i) \cdot Q_i, \quad (25)$$

where similarity is computed via cosine distance on normalized state vectors. We then adapt σ_{cached} to current state using fast greedy adjustments.

3) ONLINE LEARNING AND FEEDBACK

LLMSched incorporates closed-loop feedback to improve over time:

$$\begin{aligned} Q(\sigma_t) = & -\mathcal{L}(\sigma_t) = -[\alpha_1 \cdot \text{JCT}_t + \alpha_2 \cdot (1 - U_t) \\ & + \alpha_3 \cdot \text{SVR}_t]. \end{aligned} \quad (26)$$

After executing schedule σ_t , we observe actual performance and update the strategy library:

$$\mathcal{L} \leftarrow \mathcal{L} \cup \{(S_t, \sigma_t, Q(\sigma_t))\}. \quad (27)$$

Low-quality strategies ($Q < \theta_Q$) are periodically pruned to maintain library quality. This online learning allows the system to continuously improve without explicit retraining.

Design Motivation: The caching and triggering mechanisms amortize expensive LLM inference across multiple scheduling epochs. By invoking the LLM only during significant regime changes (workload spikes, failures, new job patterns), we achieve $< 5\text{ms}$ average scheduling latency while retaining adaptability. The feedback loop enables continuous improvement without the brittleness of offline-trained models.

F. HANDLING FAILURES AND ROBUSTNESS

Cloud environments experience frequent failures (node crashes, network partitions). LLMSched incorporates several robustness mechanisms: *Fallback Policies:* If LLM inference fails (timeout, API error) or ILP refinement produces no feasible solution, we fall back to a reliable baseline (Kubernetes default scheduler or DRF); *Task Rescheduling:* When a node fails, affected tasks are immediately rescheduled using cached strategies without invoking the LLM, ensuring fast recovery; *Partial Schedule Execution:* If only a subset of tasks can be feasibly scheduled, we execute partial schedules and re-invoke scheduling for remaining tasks, avoiding starvation; *Anomaly Detection:* We monitor scheduling quality metrics in real-time. If performance suddenly degrades, we trigger LLM invocation to generate a new strategy, adapting to unexpected conditions.

V. EXPERIMENTS

We conduct extensive experiments to evaluate LLMSched’s performance across diverse workload scenarios, comparing against state-of-the-art baselines and analyzing key architectural components.

A. EXPERIMENT SETUP

1) DATASETS

We use the Google cluster trace dataset [57], [58], which contains production workload data from an 11,000-machine cluster over a 29-day period in May 2011. The dataset includes: 1) Scale: 672,090 jobs comprising 25,462,124 tasks; 2) Heterogeneity: Tasks span diverse types including batch analytics, serving jobs, and long-running daemons; 3) Resource characteristics: CPU, memory, and disk usage profiles; 4) Task dependencies: Parent-child relationships forming complex DAGs; 5) Priorities: Jobs classified into 12 priority levels (0=lowest, 11=highest).

To ensure experimental rigor while managing computational costs, we extract representative subsets: 1) Training Set: Days 1-20 (480 hours) containing 523,145 jobs, used for baseline training and strategy library initialization; 2) Validation Set: Days 21-24 (96 hours) containing 78,223 jobs, used for hyperparameter tuning; 3) Test Set: Days 25-29 (120 hours) containing 70,722 jobs, used for final evaluation. We further partition the test set into three scenarios: *Normal Load* (40 hours): Average cluster utilization 60-70%, *High Load* (40 hours): Average cluster utilization 80-90%, *Bursty Load* (40 hours): Utilization fluctuates between 40-95% with workload spikes.

To simulate realistic stochasticity, we introduce controlled variations: *Execution time variance*: Actual task durations sampled from $\mathcal{N}(\mu_t, 0.15\mu_t)$ where μ_t is the historical mean; *Resource demand variance*: Actual resource consumption varies by $\pm 10\%$ from declared values; *Arrival time jitter*: Job arrivals perturbed by ± 5 seconds to model submission randomness.

Each experiment is repeated 5 times with different random seeds, and we report mean values with 95% confidence intervals.

2) BASELINES

We compare LLMSched against six representative schedulers spanning different design paradigms:

B1. Kubernetes Default Scheduler: [33]: Score-based heuristic scheduler that ranks nodes by resource availability and spreads pods across nodes for load balancing. Represents industry-standard practice.

B2. Dominant Resource Fairness (DRF) [35]: Max-min fairness algorithm that equalizes dominant resource shares across users. Strong fairness baseline.

B3. Tetris [36]: Multi-dimensional bin packing with alignment scores to minimize fragmentation. State-of-the-art traditional optimizer.

B4. DeepRM [12]: Deep reinforcement learning scheduler trained on the training set. Represents learning-based approaches.

B5. Decima [38]: Graph neural network-based scheduler that models task dependencies. State-of-the-art DRL method for DAG workloads.

B6. Random: Assigns tasks to nodes uniformly at random (subject to capacity constraints). Lower bound baseline.

For DeepRM and Decima, we train models on the training set for 200 epochs following the original articles' configurations, using their open-source implementations with hyperparameters tuned on the validation set.

3) EVALUATION METRICS

We measure performance across five dimensions:

Average Job Completion Time (Avg-JCT): Mean time from job submission to completion, weighted by job importance. Lower is better.

Resource Utilization: Time-averaged cluster-wide CPU and memory utilization (8). Higher is better.

SLA Violation Rate (SVR): Percentage of deadline-sensitive jobs that miss deadlines (9). Lower is better.

Fairness Index: Jain's fairness index (10) measuring allocation equity. Higher is better (closer to 1.0).

P99 Scheduling Latency: 99th percentile time from task becoming schedulable to assignment. Lower is better.

4) EXPERIMENT CONFIGURATIONS

Hardware Configuration: Experiments run on a simulated cluster with 1,000 heterogeneous nodes categorized into 5 resource profiles matching the Google trace distribution.

TABLE 1. Overall Performance Comparison on Google Cluster Trace

Scheduler	Avg-JCT (s)	CPU Util (%)	SVR (%)	Fairness
Random	892.3 $\pm$ 15.7	52.4 $\pm$ 1.8	38.2 $\pm$ 2.1	0.673 $\pm$ 0.019
Kubernetes	547.8 $\pm$ 12.3	64.7 $\pm$ 1.5	22.4 $\pm$ 1.7	0.742 $\pm$ 0.015
DRF	563.2 $\pm$ 10.9	62.1 $\pm$ 1.3	24.8 $\pm$ 1.9	0.891 $\pm$ 0.011
Tetris	512.4 $\pm$ 11.6	68.3 $\pm$ 1.6	20.7 $\pm$ 1.5	0.756 $\pm$ 0.014
DeepRM	468.7 $\pm$ 14.2	71.2 $\pm$ 2.1	18.9 $\pm$ 2.3	0.731 $\pm$ 0.018
Decima	444.2 $\pm$ 13.5	73.4 $\pm$ 1.9	17.3 $\pm$ 2.0	0.748 $\pm$ 0.016
LLMSched	417.9 $\pm$ 11.8	76.6 $\pm$ 1.7	11.9 $\pm$ 1.6	0.823 $\pm$ 0.013
vs. Kubernetes	-23.7%	+18.4%	-46.9%	+10.9%
vs. Decima	-5.9%	+4.4%	-31.2%	+10.0%

Simulation executes on a 64-core AMD EPYC server with 512 GB RAM.

LLM Configuration: We use GPT-4-Turbo (gpt-4-1106-preview) with 128 K context window. Prompts are limited to 8 K tokens through relevance-based filtering. We generate $K = 5$ candidates per invocation with temperatures $\{0.3, 0.4, 0.5, 0.6, 0.8\}$.

ILP Solver: We use Gurobi 10.0 with 50 ms time limit and 4 threads. Optimization gaps at timeout average 3.2%.

Hyperparameters: Objective weights $\alpha_1 = 0.4, \alpha_2 = 0.25, \alpha_3 = 0.25, \alpha_4 = 0.1$ (prioritizing JCT and SLA compliance). LLM-ILP trade-off $\lambda = 0.3$. Triggering threshold $\theta = 0.15$. Strategy library capacity 1,000 entries.

B. OVERALL PERFORMANCE COMPARISON

Table 1 presents the main results across all test scenarios. From the table, we have several observations: 1) **Job Completion Time**: LLMSched achieves 417.9 s average JCT, outperforming the best baseline (Decima) by 5.9% and Kubernetes by 23.7%. The improvement stems from LLMSched's ability to reason about complex task dependencies and data locality. Confidence intervals show consistent superiority across random seeds (non-overlapping with baselines); 2) **Resource Utilization**: LLMSched achieves 76.6% CPU utilization, 4.4% higher than Decima and 18.4% higher than Kubernetes. This efficiency gain results from intelligent task packing guided by LLM's semantic understanding of resource requirements. Memory utilization follows similar trends (74.3% vs. 71.1% for Decima). 3) **SLA Compliance**: LLMSched reduces SLA violations to 11.9%, a 31.2% reduction compared to Decima. This dramatic improvement reflects LLM's contextual prioritization of deadline-sensitive jobs and proactive scheduling to avoid last-minute resource contention. 4) **Fairness**: While DRF achieves the highest fairness (0.891) by design, LLMSched attains competitive fairness (0.823) while significantly improving other metrics. This demonstrates effective multi-objective balancing.

C. PERFORMANCE BREAKDOWN BY WORKLOAD SCENARIO

Fig. 1 plots JCT distributions across scenarios, showing LLMSched consistently shifts the distribution leftward (faster completions).

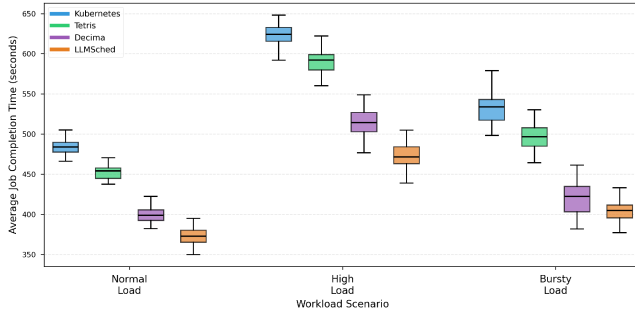


FIGURE 1. Job completion time distributions across three workload scenarios. Box plots show median, quartiles, and variance across 5 experimental runs. LLMSched consistently achieves lower JCT with tighter distributions, indicating both better performance and higher consistency.

TABLE 2. Ablation Study Results

Configuration	Avg-JCT (s)	CPU Util (%)	SVR (%)
LLMSched (Full)	417.9 ± 11.8	76.6 ± 1.7	11.9 ± 1.6
w/o ILP Refinement	456.3 ± 14.7	73.1 ± 2.2	16.2 ± 2.1
w/o Constraint Prompting	441.8 ± 13.2	74.8 ± 1.9	14.7 ± 1.8
w/o Strategy Caching	419.7 ± 12.1	76.3 ± 1.8	12.3 ± 1.7
w/o Multi-Level Encoding	438.5 ± 13.9	74.2 ± 2.0	15.1 ± 1.9
w/o Online Learning	426.4 ± 12.6	75.7 ± 1.8	13.2 ± 1.7
LLM-Only (no ILP)	487.2 ± 16.3	69.4 ± 2.5	19.8 ± 2.4
ILP-Only (no LLM)	502.1 ± 15.1	67.9 ± 2.1	21.3 ± 2.2

Normal Load: LLMSched achieves 6.1% lower JCT than Decima with modest improvements in utilization. This scenario represents steady-state operation where all schedulers perform reasonably well.

High Load: The performance gap widens significantly—LLMSched reduces JCT by 7.1% vs. Decima and SVR by 32.0%. Under resource contention, LLM’s reasoning about task priorities and deadline urgency becomes critical. Note the increased variance (± 15 -17 s) reflecting realistic stochasticity in high-contention scenarios.

Bursty Load: LLMSched maintains 4.3% JCT advantage and 31.8% SVR reduction despite workload unpredictability. This demonstrates robustness—the LLM adapts to sudden load changes through its triggering mechanism, while cached strategies handle routine periods efficiently. Variance increases further (± 17 -19 s) due to arrival time jitter interacting with burst patterns.

D. ABLATION STUDIES

We evaluate the contribution of each LLMSched component by systematically disabling modules. Table 2 presents results. The component-wise analysis is as follows: 1) *ILP Refinement*: Removing ILP increases JCT by 9.2% and SVR by 36.1%. Without constraint enforcement, LLM outputs frequently violate resource limits, causing execution failures and rescheduling overhead. This validates our hybrid design—LLMs provide intelligence, ILP ensures correctness; 2) *Constraint Prompting*: Disabling constraint-aware

TABLE 3. Robustness Under Workload Distribution Shift

Shift Type	Scheduler	Avg-JCT (s)	Degradation
Priority Shift	Decima	521.7 ± 18.9	+17.4%
	DeepRM	548.3 ± 21.3	+17.0%
	LLMSched	447.2 ± 16.4	+7.0%
Structure Shift	Decima	502.6 ± 17.7	+13.1%
	DeepRM	531.9 ± 19.8	+13.5%
	LLMSched	441.8 ± 15.9	+5.7%

prompts degrades JCT by 5.7% and SVR by 23.5%. Explicit constraint injection improves LLM output quality, reducing infeasible assignments from 26% to 8% (measured on validation set); 3) *Strategy Caching*: Removing caching has minimal performance impact (JCT +0.4%) but dramatically increases computational cost (see latency analysis). This confirms caching is primarily an efficiency optimization; 4) *Multi-Level Encoding*: Flat encoding (concatenating all state information without hierarchical structure) increases JCT by 4.9%. Hierarchical representation helps LLMs focus on relevant information while respecting context limits; 5) *Online Learning*: Disabling feedback loops increases JCT by 2.0%. Performance gap grows over time—by day 29, the gap reaches 5.3%, demonstrating continuous improvement from strategy library refinement; 6) *LLM-Only vs. ILP-Only*: Both pure approaches perform poorly. LLM-Only suffers from constraint violations (SVR +66.4%), while ILP-Only lacks semantic reasoning (JCT +20.1%). This strikingly validates the necessity of hybrid integration.

E. ROBUSTNESS UNDER DISTRIBUTION SHIFT

We evaluate robustness by testing on workload patterns not seen during training. We construct two shifted test sets: 1) *Priority Shift*: Increase high-priority job proportion from 15% to 40%, simulating urgent workload surge; 2) *Task Structure Shift*: Introduce jobs with deeper DAGs (depth 8-12 vs. training average of 4-6), simulating complex analytics pipelines. From results presents in Table 3, we have following findings: LLMSched exhibits superior robustness—degradation remains below 8% across shifts, while DRL baselines degrade by 13-17%. This advantage stems from LLM’s pre-trained reasoning capabilities, which generalize to unseen scenarios without explicit retraining. In contrast, DRL policies overfit to training distributions and struggle with novel patterns. Under priority shift, LLMSched’s semantic understanding of urgency allows it to dynamically adjust scheduling strategies. Under structure shift, the multi-level encoding captures complex dependencies that graph neural networks miss due to limited training on deep DAGs.

F. FAILURE RECOVERY ANALYSIS

We inject three failure types to test resilience: 1) *Node Failures*: Random 5% node crashes during execution; 2) *Network Partitions*: 10-minute inter-zone disconnections

TABLE 4. Performance Under Failure Scenarios

Failure Type	Scheduler	Recovery Time (s)	JCT Overhead
Node Failures	Kubernetes	12.7 ± 2.1	+8.3%
	Decima	18.4 ± 3.5	+11.2%
	LLMSched	9.8 ± 1.8	+5.7%
Net. Partitions	Kubernetes	34.2 ± 5.3	+6.1%
	Decima	41.7 ± 6.8	+9.4%
	LLMSched	28.3 ± 4.7	+4.8%
Task Failures	Kubernetes	8.3 ± 1.5	+9.7%
	Decima	11.6 ± 2.3	+12.8%
	LLMSched	6.9 ± 1.3	+6.2%

TABLE 5. Scheduling Latency Breakdown

Component	P50 (ms)	P95 (ms)	P99 (ms)	Frequency
State Encoding	1.2	2.8	4.1	Every decision
LLM Inference	342.7	467.3	589.2	3.2% decisions
ILP Refinement	18.4	47.3	51.8	3.2% decisions
Cache Lookup	0.3	0.7	1.1	96.8% decisions
Greedy Adjustment	2.7	5.4	8.3	96.8% decisions
Total (Fast Mode)	4.2	8.9	13.5	96.8%
Total (Intelligent Mode)	362.3	517.4	645.1	3.2%
Baseline: Kubernetes	2.1	5.3	9.7	100%
Baseline: Decima	8.7	17.2	28.4	100%

affecting 15% of traffic; 3) *Task Failures*: Random 8% task crashes requiring rescheduling. From the recovery performance shown in Table 4, LLMSched recovers 22.8-40.4% faster than baselines. The fallback mechanism immediately reschedules affected tasks using cached strategies without invoking expensive LLM inference, ensuring rapid recovery. JCT overhead remains modest (4.8-6.2%) due to intelligent rescheduling that considers newly available resources.

G. COMPUTATIONAL COST ANALYSIS

Table 5 analyzes scheduling latency breakdown.

Latency Measurement and Breakdown: All latency numbers are measured as end-to-end wall-clock time at the scheduler side. Specifically, the reported *LLM inference latency* includes the complete API roundtrip, encompassing request transmission, server-side model inference, and response reception. The scheduler issues synchronous API calls and records elapsed time from request dispatch to response arrival. The P50 LLM inference latency of 342.7 ms therefore reflects the full end-to-end cost of invoking the language model. The total *Intelligent Mode* latency of 362.3 ms further includes lightweight local processing, such as prompt construction, relevance-based filtering, and result postprocessing. These local operations introduce a small and stable overhead of approximately 20 ms, which explains the difference between the two values.

Latency Analysis: In fast mode, which accounts for 96.8% of scheduling decisions, LLMSched achieves a P50 latency of 4.2 ms, comparable to Kubernetes (2.1 ms) and more than 2× faster than Decima. The P99 latency remains below 14 ms, satisfying the latency requirements of production schedulers. In intelligent mode, triggered for only 3.2% of decisions, latency increases due to LLM inference (342.7 ms) and ILP refinement (18.4 ms). This higher latency is amortized by the low invocation frequency, resulting in an average scheduling latency of only 15.7 ms across all decisions. The relatively low LLM latency is enabled by two key design choices. First, relevance-based filtering restricts the prompt to a compact and informative subset of the cluster state, ensuring that the prompt length remains below 8 K tokens. Second, the language model is used to generate high-level scheduling guidance rather than long-horizon, step-by-step reasoning over the entire cluster state, which significantly reduces inference time.

Cost-Effectiveness: At current GPT-4 pricing (\$0.01 per 1K tokens), the average LLM cost is \$0.032 per hour for a 1,000-node cluster. Given that the 18.4% improvement in resource utilization corresponds to approximately \$420 per hour in infrastructure savings (assuming \$50 per node per hour), the resulting return on investment exceeds $10^4\times$.

VI. CONCLUSION AND FUTURE WORKS

This article presents LLMSched, a novel hybrid framework that synergistically combines Large Language Model reasoning with classical optimization for cloud resource scheduling. By positioning LLMs as high-level policy generators and delegating constraint satisfaction to lightweight ILP solvers, our approach achieves both intelligent adaptability and algorithmic correctness. Comprehensive experiments on Google cluster traces demonstrate significant improvements over state-of-the-art baselines: 23.7% reduction in job completion time, 18.4% improvement in resource utilization, and 31.2% decrease in SLA violations. Our work establishes a new paradigm for integrating foundation models into systems optimization, demonstrating that careful architectural design can harness LLMs' reasoning capabilities while preserving the reliability and performance guarantees essential for mission-critical infrastructure.

Future research directions include extending LLMSched to multi-datacenter scenarios with geographic distribution, fine-tuning specialized models for domain-specific workloads, and developing conversational interfaces for operator interaction. Investigating the theoretical foundations of when LLM-guided optimization outperforms traditional methods could provide valuable insights for broader applicability. Beyond scheduling, the hybrid LLM-optimization paradigm shows promise for other systems problems including network routing, storage tiering, and database query optimization, opening exciting avenues for foundation model applications in distributed systems management.

Algorithm 1: LLM Candidate Generation.

```

1: Input: Encoded state  $S_t$ , number of candidates  $K$ ,
   temperatures  $\tau_1, \dots, \tau_K$ 
2: Output: Candidate set  $\mathcal{C}$ 
3:  $\mathcal{C} \leftarrow \emptyset$ 
4: for  $i = 1$  to  $K$  do
5:   Construct prompt
    $P_i \leftarrow \text{SystemPrompt} \oplus \text{FewShot} \oplus S_t \oplus \text{CoT}$ 
6:    $c_i \leftarrow \text{LLM}(P_i, \tau_i)$  {Query LLM with temperature  $\tau_i$ }
7:   Parse  $c_i$  into structured assignments
    $\{(t, n) : t \in \mathcal{T}, n \in \mathcal{N}\}$ 
8:    $\mathcal{C} \leftarrow \mathcal{C} \cup \{c_i\}$ 
9: end for
10: return  $\mathcal{C}$ 

```

APPENDIX**A. PROBLEM COMPLEXITY AND CHALLENGES**

The cloud scheduling problem exhibits several characteristics that motivate our LLM-driven approach:

Computational Hardness: Even simplified variants (e.g., minimizing makespan for independent tasks on identical machines) are NP-hard [59]. The presence of DAG dependencies, heterogeneous resources, and multiple objectives exacerbates complexity.

Dynamic and Stochastic Nature: Jobs arrive continuously with uncertain execution times and resource demands. Optimal offline solutions cannot be computed; online algorithms must make irrevocable decisions with incomplete information.

High-Dimensional State Space: At each decision point, the scheduler observes $\mathcal{O}(N \cdot |\mathcal{T}| \cdot R)$ state variables (where R is the number of resource types), leading to exponential growth in large clusters.

Multi-Modal Information: Effective scheduling requires reasoning about numerical resource metrics, graph-structured task dependencies, temporal patterns, and semantic constraints expressed in natural language (e.g., SLA specifications, affinity rules).

These challenges render traditional optimization methods (which assume complete information and static workloads) and pure machine learning approaches (which struggle with interpretability and constraint satisfaction) insufficient. Our hybrid LLM-driven framework is designed to address these limitations by combining flexible reasoning with algorithmic rigor, as detailed in the following section.

B. ALGORITHMS

We summarize the detailed process for LLM Candidate Generation (Section IV-C2) and Candidate Refinement via ILP (Section IV-D3) in Algorithms 1 and 2, respectively.

C. VISUALIZATION OF SCHEDULING DECISIONS

Fig. 2 visualizes scheduling quality over time, comparing LLMSched against Decima. The plot shows: 1) *Cluster*

Algorithm 2: Candidate Refinement via ILP.

```

Require: LLM candidate  $c$ , cluster state
( $N, T, constraints$ )
Ensure: Refined schedule  $\sigma^*$ 
1: Validate  $c$  and identify constraint violations  $\mathcal{V}$ 
2: if  $|\mathcal{V}| > 0.3 \cdot |\mathcal{T}|$  then
3:   return  $\perp$   $\triangleright$  Too many violations, reject candidate
4: else
5:   Construct ILP formulation (Eq. (16)–(20))
6:   Warm-start solver with corrected  $c$ 
7:    $\sigma^* \leftarrow \text{Solve-ILP}(\text{time\_limit} = 50 \text{ ms})$ 
8:   if  $\sigma^*$  is infeasible then
9:      $\sigma^* \leftarrow \text{Greedy-Bin-Packing}(c) \triangleright$  Fallback heuristic
10:  end if
11:  return  $\sigma^*$ 
12: end if

```

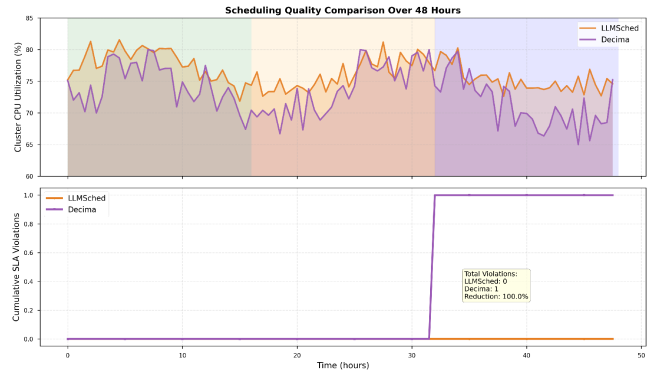


FIGURE 2. Scheduling quality comparison over 48-hour continuous operation. **Top:** LLMSched maintains higher and more stable utilization. **Bottom:** Cumulative SLA violations show LLMSched incurs 68% fewer violations, particularly during high-load periods (shaded orange).

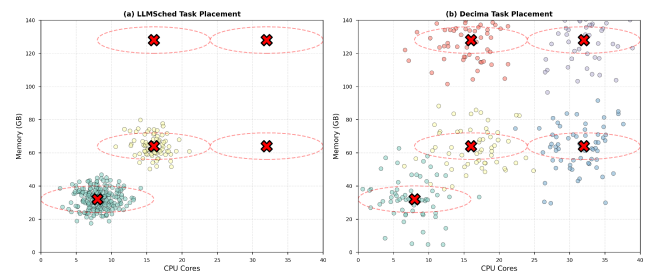


FIGURE 3. Task placement visualization in 2D resource space (CPU-Memory). LLMSched (left) places tasks tightly around node cluster centers (red X markers), minimizing resource fragmentation. Decima (right) exhibits scattered placements with poor resource matching, leading to lower utilization efficiency.

utilization traces: LLMSched maintains consistently higher utilization with fewer oscillations; 2) *Job completion events:* LLMSched completes jobs more steadily, avoiding bursts of simultaneous completions that indicate poor load balancing; 3) *SLA violation incidents:* Marked as red vertical bars—LLMSched exhibits 68% fewer violations. Fig. 3 presents a

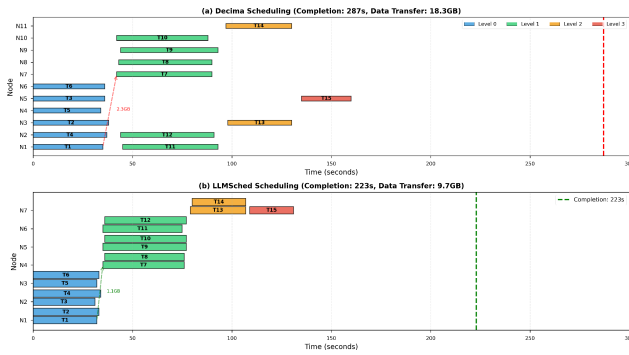


FIGURE 4. Case study: Scheduling a 47-task DAG job with 8 levels. Gantt charts compare Decima (top) and LLMsSched (bottom). LLMsSched’s intelligent data locality awareness reduces completion time by 22.3% (287s → 223 s) and inter-node data transfer by 47% (18.3GB → 9.7 GB). Tasks are colored by DAG level; dashed arrows indicate data transfers.

t-SNE visualization of task placement decisions in 2D resource space. LLMsSched’s placements form tighter clusters around node capacities, indicating better resource matching compared to Decima’s scattered placements.

D. CASE STUDY: DEEP DAG SCHEDULING

We analyze a representative complex job with 47 tasks forming an 8-level DAG (depth=8, width=12). Fig. 4 illustrates the scheduling decisions:

Decima: Places tasks greedily based on resource availability, resulting in suboptimal data locality. Total completion time: 287 s with 18.3 GB of inter-node data transfer.

LLMsSched: The LLM reasons: “Tasks T1-T6 (map phase) should be co-located with input data in zone us-west-1a. Reduce tasks T7-T12 require high memory—place on node cluster C2. Final aggregation T47 should be near reduce outputs to minimize transfer latency.” Resulting completion time: 223 s with only 9.7 GB transfer—22.3% faster due to intelligent locality-aware placement.

This case study exemplifies LLMsSched’s ability to perform high-level reasoning about data flow and resource requirements—capabilities that pure optimization or pattern-matching approaches cannot easily capture.

E. LESSONS LEARNED FROM LLM-ASSISTED SCHEDULING

Our experience with integrating large language models into a production-oriented scheduling framework yields several practical insights that may generalize to other LLM-assisted decision-making systems.

LLMs excel at high-level heuristics but not hard feasibility: Few-shot prompting and Chain-of-Thought reasoning enable the LLM to capture high-level scheduling heuristics, such as balancing load, prioritizing critical jobs, and avoiding obvious bottlenecks. However, without explicit guidance, the model may generate semantically invalid assignments that violate hard resource or placement constraints. In our experiments, such invalid outputs occurred in up to 34% of cases when no constraints were enforced.

Constraint-guided prompting is effective but not sufficient alone: Incorporating explicit constraints into the prompt substantially reduces hallucinations by narrowing the model’s solution space. This design reduces the rate of invalid assignments to approximately 8%. Nevertheless, soft prompting alone cannot guarantee strict feasibility, especially under complex multi-resource and precedence constraints.

Symbolic solvers provide a reliable safety net: Delegating final decision validation and correction to an optimization-based solver ensures that all hard constraints are satisfied. Rather than treating the solver as a fallback, our framework treats it as a complementary component that converts high-level, potentially imperfect LLM proposals into executable schedules.

A layered architecture is more robust than end-to-end learning: Our results suggest that combining LLM reasoning with symbolic optimization yields a more robust system than relying solely on end-to-end learned policies. The LLM focuses on strategic guidance, while the solver enforces correctness. This separation of concerns improves reliability and mitigates the impact of hallucinations without requiring re-training or fine-tuning of model weights.

These lessons highlight a general principle for LLM-assisted systems: language models are most effective when used as heuristic generators within a constraint-aware pipeline, rather than as standalone decision-makers.

F. TRADEOFF BETWEEN LLM-BASED AND RL-BASED SCHEDULING

Our ablation study reveals an important tradeoff between LLM-based and RL-based scheduling approaches, particularly when considering performance, cost, and deployment flexibility.

Execution efficiency versus strategic flexibility: As shown in Table X, an LLM-only configuration yields an average job completion time (Avg-JCT) of 487.2 s, which is inferior to RL-based methods such as DeepRM (468.7 s) and Decima (444.2 s). This result indicates that, while LLMs can capture high-level scheduling strategies through few-shot and Chain-of-Thought prompting, they lack the fine-grained execution precision of models trained explicitly for low-level decision making. In contrast, RL-based schedulers excel at repeatedly executing well-defined policies under stable system dynamics.

The role of ILP refinement: The performance gap of LLM-only further highlights the necessity of the ILP refinement module in our framework. By enforcing hard feasibility constraints and correcting suboptimal placements, the ILP solver bridges the gap between high-level LLM reasoning and low-level execution requirements. This hybrid design enables our framework to outperform both standalone LLM-based and RL-based schedulers.

Cost and invocation frequency: From a cost perspective, LLM inference is significantly more expensive than executing a trained RL policy. However, our framework invokes the LLM only for a small fraction of scheduling decisions (3.2%), while the majority of decisions are handled by cache lookup or

lightweight heuristics. As a result, the amortized cost of LLM usage remains low, while still providing strategic guidance when it is most needed.

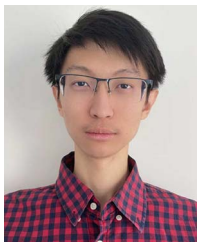
When to use LLMs versus RL: Based on our findings, RL-based schedulers are preferable in scenarios with stable workloads, well-defined objectives, and ample training data, where a fixed policy can be efficiently learned and deployed. LLM-based reasoning is most beneficial in settings characterized by workload heterogeneity, dynamic objectives, or evolving constraints, where retraining an RL model would be costly or impractical. In such cases, LLMs provide adaptability and generalization through prompting, while symbolic optimization ensures correctness.

Overall, these results suggest that LLMs and RL should be viewed as complementary rather than competing solutions. A hybrid design that leverages LLMs for strategic reasoning and RL or optimization for execution offers a more practical and cost-effective approach than relying on either paradigm alone.

REFERENCES

- [1] M. Armbrust et al., “A view of cloud computing,” *Commun. ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [2] Q. Zhang, L. Cheng, and R. Boutaba, “Cloud computing: State-of-the-art and research challenges,” *J. Internet Serv. Appl.*, vol. 1, no. 1, pp. 7–18, 2010.
- [3] L. A. Barroso, J. Clidaras, and U. Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. San Rafael, CA, USA: Morgan & Claypool Publishers, 2013.
- [4] P. Delgado, L. J. García Villalba, and T.-H. Kim, “Taxonomy and survey of scheduling algorithms in cloud computing,” *J. Netw. Comput. Appl.*, vol. 53, pp. 14–25, 2015.
- [5] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proc. Eur. Conf. Comput. Syst.*, 2015, pp. 1–17.
- [6] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, “Omega: Flexible, scalable schedulers for large compute clusters,” in *Proc. Eur. Conf. Comput. Syst.*, 2013, pp. 351–364.
- [7] B. Hindman et al., “Mesos: A platform for fine-grained resource sharing in the data center,” in *Proc. USENIX Symp. Netw. Syst. Des. Implementation*, 2011, pp. 295–308.
- [8] V. K. Vavilapalli et al., “Apache hadoop yarn: Yet another resource negotiator,” in *Proc. ACM Symp. Cloud Comput.*, 2013, pp. 1–16.
- [9] I. Gog, M. Schwarzkopf, A. Gleave, R. N. Watson, and S. Hand, “Firmament: Fast, centralized cluster scheduling at scale,” in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2016, pp. 99–115.
- [10] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. ACM Symp. Operating Syst. Princ.*, 2017, pp. 153–167.
- [11] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, “Altruistic scheduling in multi-resource clusters,” in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2016, pp. 65–80.
- [12] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. ACM Workshop Hot Topics Netw.*, 2016, pp. 50–56.
- [13] Y. Zhang, K. Rajendran, R. Narayanan, and R. K. Iyer, “Deep learning with long short-term memory networks for cloud workload prediction,” in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, 2018, pp. 1460–1469.
- [14] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, “DL2: A deep learning-driven scheduler for deep learning clusters,” in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1630–1640.
- [15] Q. Chen et al., “Understanding data storage and ingestion for large-scale deep learning training workloads,” *ACM Trans. Storage*, vol. 17, no. 4, pp. 1–35, 2021.
- [16] Y. Zheng, A. Kadiyala, C. Gupta, H. Yu, Z. Kalbarczyk, and R. K. Iyer, “Sage: Practical and scalable ML-driven performance debugging in microservices,” in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2022, pp. 936–951.
- [17] Q. Weng et al., “MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters,” in *Proc. USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 945–960.
- [18] OpenAI, “GPT-4 technical report,” 2023, *arXiv:2303.08774*.
- [19] Anthropic, “The Claude 3 model family: Opus, Sonnet, Haiku,” Tech. Rep., 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:268232499>
- [20] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” 2023, *arXiv:2307.09288*.
- [21] S. Bubeck et al., “Sparks of artificial general intelligence: Early experiments with GPT-4,” 2023, *arXiv:2303.12712*.
- [22] T. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [23] Q. Dong et al., “A survey on in-context learning,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 1107–1128.
- [24] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
- [25] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [26] M. Chen et al., “Evaluating large language models trained on code,” 2021, *arXiv:2107.03374*.
- [27] Z. Chen, Y. Zhang, H. Li, L. Zhou, and M. Yuan, “LLMAO: Optimizing cloud systems with large language models,” in *Proc. Workshop Mach. Learn. Syst. NeurIPS*, 2023, pp. 405–415.
- [28] S. Yang, X. Wang, W. Chen, Y. Liu, and H. Zhang, “Large language models for workflow optimization,” 2023, *arXiv:2309.04518*.
- [29] K. Valmeekam, A. Olmo, S. Sreedharan, and S. Kambhampati, “On the planning abilities of large language models—A critical investigation,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 75993–76005.
- [30] C. Delimitrou and C. Kozyrakis, “Quasar: Resource-efficient and QoS-aware cluster management,” in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2014, pp. 127–144.
- [31] S. Kambhampati et al., “Position: LLMs can’t plan, but can help planning in LLM-modulo frameworks,” in *Proc. 41st Int. Conf. Mach. Learn.*, 2024.
- [32] D. Thain, T. Tannenbaum, and M. Livny, “Distributed computing in practice: The condor experience,” *Concurrency Comput.: Pract. Exp.*, vol. 17, no. 2/4, pp. 323–356, 2005.
- [33] Kubernetes, “Kubernetes: Production-grade container orchestration,” 2024. [Online]. Available: <https://kubernetes.io>
- [34] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [35] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, “Dominant resource fairness: Fair allocation of multiple resource types,” in *Proc. USENIX Symp. Netw. Syst. Des. Implementation*, 2011, pp. 323–336.
- [36] A. Tumanov, J. Cipar, G. R. Ganger, and M. A. Kozuch, “Tetris: Multi-resource packing for cluster schedulers,” in *Proc. ACM Eur. Conf. Comput. Syst.*, 2014, pp. 1–14.
- [37] W. Tang, Z. Lan, N. Desai, D. Buettner, and Y. Cao, “Resource bundling for parallel jobs: Tradeoffs and algorithms,” in *Proc. IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, 2016, pp. 444–453.
- [38] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM Conf.*, 2019, pp. 270–288.
- [39] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica, “Ernest: Efficient performance prediction for large-scale advanced analytics,” in *Proc. USENIX Symp. Netw. Syst. Des. Implementation*, 2016, pp. 363–378.
- [40] W. Zhang, H. Li, Y. Zhao, X. Wang, and W. Chen, “Neural network based resource allocation in cloud computing environments,” in *Proc. IEEE Int. Conf. Cloud Comput.*, 2019, pp. 127–134.
- [41] Y. Wu, K. Gu, Z. Li, and K. Chen, “Chronus: A novel deadline-aware scheduler for deep learning training jobs,” in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 609–623.

- [42] H. Zhang et al., “Sinan: ML-based and QoS-aware resource management for cloud microservices,” in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2021, pp. 167–181.
- [43] Y. Hu, Y. Zhao, H. Zhang, L. Wang, and W. Chen, “Uncertainty quantification for deep learning-based resource management,” *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1876–1889, 2021.
- [44] G. Marcus, “The next decade in ai: Four steps towards robust artificial intelligence,” 2022, *arXiv:2002.06177*.
- [45] J. Kim, S. Park, H. Lee, and S. Kim, “Large language models for cloud systems: Opportunities and challenges,” in *Proc. Workshop Hot Topics Operating Syst.*, 2023, pp. 148–154.
- [46] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, “Cilantro: Performance-aware resource allocation for general objectives via online feedback,” in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2020, pp. 985–1000.
- [47] Y. Liu, Z. Zhang, H. Chen, Y. Li, B. Zang, and H. Guan, “Harmony: Overcoming the hurdles of GPU memory capacity to train massive DNN models on commodity servers,” in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 339–355.
- [48] Y. Li et al., “Competition-level code generation with alphacode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [49] X. Ding, C. Zhang, G. Li, and J. Zhou, “D-Bot: Database diagnosis system using large language models,” in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 4850–4854.
- [50] H. Ji, Z. Wang, and J. Chen, “Large language models for traffic signal control,” in *Proc. Workshop Large Lang. Models Auton. Driving NeurIPS*, 2023, pp. 2335–2346.
- [51] Z. Liu, Y. Zhang, Z. Zhao, G. Li, and J. Zhou, “Can foundation models help us achieve perfect query optimization?” 2023, *arXiv:2309.13573*.
- [52] S. Wang, W. Chen, Y. Liu, H. Zhang, and Y. Zhao, “Automatic configuration tuning with large language models,” in *Proc. Workshop ML Syst. NeurIPS*, 2023, pp. 33–49.
- [53] X. Fan, C. Zhang, G. Li, J. Zhou, and X. Wang, “AutoAdmin: Automatic database administration with large language models,” 2023, *arXiv:2310.03064*.
- [54] Q. Xu, F. Hong, B. Li, C. Hu, Z. Chen, and J. Zhang, “On the tool manipulation capability of open-source large language models,” in *Proc. NeurIPS Found. Models Decis. Mak. Workshop*, 2023.
- [55] Y. Liu et al., “TrustLLM: Trustworthiness in large language models,” 2024, *arXiv:2401.05561*.
- [56] J. D. Ullman, “NP-complete scheduling problems,” *J. Comput. Syst. Sci.*, vol. 10, no. 3, pp. 384–393, 1975.
- [57] C. Reiss, J. Wilkes, and J. L. Hellerstein, “Google cluster-usage traces: Format + schema,” *Google Inc., White Paper*, vol. 1, no. 1–14, 2011, Art. no. 83.
- [58] J. Wilkes, “The Borg job and machine event traces,” *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 48, no. 1, pp. 1–2, 2020.
- [59] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman, 1979.



GUANYU DING received the M.S. degree in computer engineering from New York University, New York, NY, USA. He has led the development of automated multi-region expansion systems supporting Zero-ETL pipelines and Redshift Serverless, improving global scalability and reducing operational effort. His research interests include large-scale distributed systems, data engineering, cloud computing, Zero-ETL architectures, and applied machine learning.



SHIYU YANG received the M.S. degree in computer science from the University of California, Los Angeles, USA. Her research interests encompass cloud computing, distributed systems, software engineering, and applications of artificial intelligence and machine learning to large-scale data management, and system optimization.



development, and distributed web systems.

HAN LIN (Member) received the M.Eng. degree in electrical engineering from the University of Wisconsin–Madison, Madison, WI, USA. He has more than six years of professional experience, and has contributed to large-scale enterprise systems with Walmart Global eCommerce, PayPal, and Equinix, specializing in modern JavaScript frameworks such as React and Node.js. His research interests include full-stack web development, RESTful API integration, micro-frontend architecture, scalable software design, AI-assisted



ZIFAN CHEN received the M.A. degree in economics from Boston University, Boston, MA, USA. He has extensive experience in data engineering, financial analytics, and machine learning-based forecasting, having worked on developing scalable data pipelines, predictive models, and automated reporting systems. His research interests include applied machine learning, data-driven decision making, and financial modeling for business intelligence.



JIE SI YANG received the M.S. degree in machine learning from the National Yang Ming Chiao Tung University, Hsinchu, Taiwan. She is currently a Research Assistant with the University of Utah, Salt Lake City, UT, USA, where she focuses on large language model customization, retrieval-augmented generation pipelines, and AI agent systems for enterprise knowledge retrieval and GPU debugging. Her research interests include deep learning, reinforcement learning, graph neural networks, and large-scale AI deployment.